Report (Matricola 336520)

Salve cari docenti. Il codice credo sia COMPLETAMENTE sbagliato (tranne al massimo le strutture dati).
Due cose gravi da segnalare:

1 Non sono riuscito a comprendere bene il testo, nel senso che ho inteso il grafo come un albero, ma in realà, ovviamente, non è minimamente cosi.

2 So bene che la logica di utilizzo delle DFS non è questa, cioe se devo trovare la "distanza" minima tra un nodo e un altro, ovviamente, si dovrebbe usare la BFS, ma con l'ansia dell'esame non ci ho pensato (o meglio, magari si, ma è una strada che ho immediatamente accantonato in quanto non avevo idea di come implementarla bene).

Problemi principali nella prima parte (Strutture dati):
Tcnicamente è quella andata meglio. Ho dimenticato, nella KEYcmp, di ritornare 0 se anche le due date sono uguali, il resto però dovrebbe essere corretto. Nel codice corretto la data ha un suo ADT specifico e nella ST salvo un item contenente anche il nome, in questo modo è più modulare. E soprattutto, avendo più tempo, la data è una struct di interi anziche una stringa.

Problemi nella seconda parte (Verifica):

L'idea della DFS, però, magari, non è completamente errata (anche se ovviamente, molto, molto più inefficiente), in quanto si potrebbero trovare i cammini tra un vertice e un altro anche con una DFS. Bisognerebbe però assumere di partire dallo stesso antenato per ogni figlio (cioè da tutti padri, tutti nonni e cosi via). Il vettore visited, nella prima parte della funzione (non ho cambiato il nome per mancanza di tempo) rappresenta in realtà un vettore di padri, (cioe da quale vertice ho raggiunto quello che sto analizzando adesso nella DFS). Ovviamente, pero, questo non mi da nessuna info sul "livello" di profondità (e quindi di distanza) che ho tra il vertice padre di tutti e quello che sto analizzando adesso. Inoltre a me servirebbe trovare il "cammino minimo" (Non nel vero senso del termine), ma comunque, non fermandomi per tempo nella dfs (dopo aver visitato k vertici), questo non si puo fare. Io, ciclando sul vettore dopo tutta la dfs, vedo in base al "tempo" di visita e non a che distanza (tipo, se guardo prima tutto il sottoalbero sinistro e poi quello destro (e magari quello che mi serve è proprio il figlio destro) trovero quel vertice come ultimo, e questo è ovviamente sbagliato perche ritornerebbe falso).

La seconda parte della check è anch'essa sbagliata per lo stesso identico motivo. A parte l'inefficienza inaudita (perche è praticamente più che quadratica sul numero di vertici), ma soprattutto, come prima, non calcolo bene le distanze tra i vertici. Ho anche dimenticato di incrementare m, quindi il ciclo è infinito, ma comunque il vettore st, non ha la somma corretta dei vertici perche nella dfs non mi fermo a k (ma comunque, la seconda parte della check sarebbe incredibilmente complessa anche con la dfs, da implementare).

Bisognerebbe, in teoria, girare il grafo, fino ad incontrare un vertice comune, allora, vedo se la somma è minore k, allora ritorna 1. Oppure (soluzione proposta nella correzione) si fa una bfs su ogni antenato e si vede se l'antenato ha i 2 figli cercati e la somma dei cammini è minore o uguale di k.

Problema di ricerca e ottimizzazione:

I problemi nel terzo punto sono praticamente gli stessi della parte di check. Ho anche messo una len— all'interno di un ciclo che quindi non conta cosa dovrebbe (ma tanto il metodo usato è lo stesso di prima, quindi è comunque invalido). Quindi qui c'è poco da dire. Il codice non è corretto per gli stessi motivi di prima.

Chiedo scusa per questo brutto esame, ci tengo a fare sapere che non sono una persona che "prova" gli esami, (mi sono esercitato su tutti i temi d'esame su moodle più altri trovati su github fino al 2021).

È la materia che ho studiato di più in tutto il semestre (infatti la parte di teoria è andata bene).

Correzioni:

Faccio tutto con la BFS, implementando la ST e la PQ nei rispettivi .h. Questo migliora di molto l'efficienza e soprattutto è più semplice fare un codice funzionante. Anche la terza parte poteva essere fatta con una bfs (cioè senza fare una funzione per forza ricorsiva).
La ST contiene degli item che contengono anche i nomi.